
ChessTrainer – Web-Based Chess Analysis Platform



Python Chess Game with AI Opponent

- **Software Engineering Major Project**
- **Dylan Van**
- **Year 12 Software Engineering**
- **2026**

Why did I develop chess?

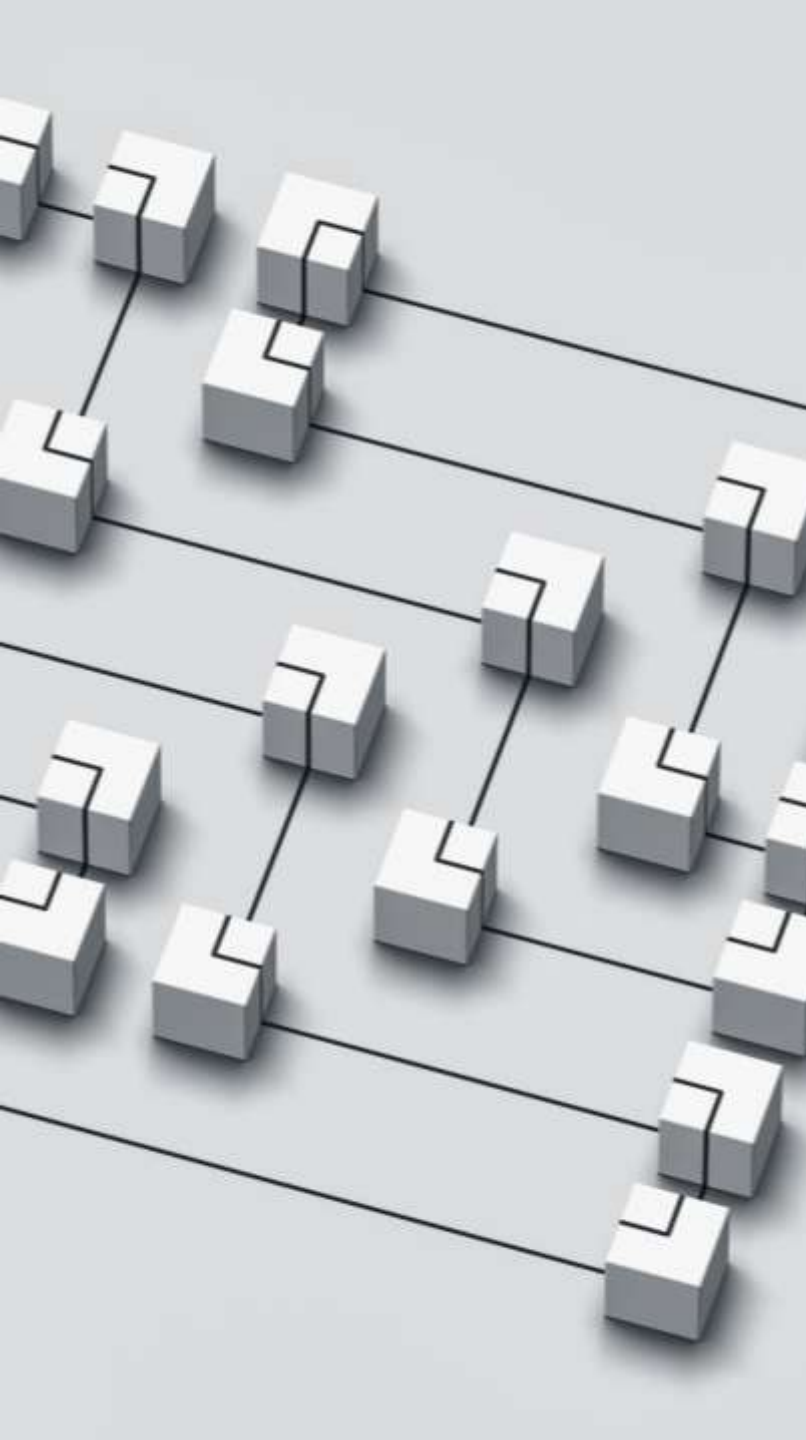


- To create an interactive chess program that helps users improve their chess skills.
- To provide beginner and intermediate players with an accessible way to practise against an AI opponent.
- To apply object-oriented programming (OOP) concepts in a real-world software project.
- To develop my Python programming skills using the Pygame library.
- To learn how artificial intelligence can be used for game decision-making.
- To create a chess program that includes modern features such as AI difficulty levels, timers, move history and game review.
- To improve my software development, debugging and problem-solving skills throughout a large-scale project.
- To produce a complete software solution that demonstrates planning, design, implementation, testing and evaluation.



Benefits to users

- Improves strategic thinking and decision-making skills.
- Helps users learn the rules of chess through regular practice.
- Allows players to improve at their own pace using different AI difficulty levels.
- Provides unlimited practice without requiring another person to play.
- Gives instant feedback through legal move validation and game outcomes.
-
- Encourages logical thinking and forward planning.
- Includes timed game modes to improve time management during chess games.
- Allows users to review completed games to identify mistakes and improve future performance.
- Easy to use with a simple and intuitive interface.
- Can be used anywhere without an internet connection.



Benefits to me as a developer

- Improved my Python programming skills.
- Gained experience with object-oriented programming principles.
- Learned how AI algorithms are implemented in games.
- Developed debugging and testing skills by fixing real programming issues.
- Improved project management through planning, documentation and version control.
- Strengthened problem-solving and logical thinking abilities.
-
- Learned to design user-friendly graphical interfaces with Pygame.
- Gained experience working on a long-term software engineering project.

Technical benefits of the software

- Automatic AI opponent provides continuous gameplay.
- Multiple difficulty levels cater to different player abilities.
- Timers simulate real competitive chess.
- Move history allows post-game analysis.
- Planning arrows help users visualise strategies.
- Click-to-move controls improve accessibility.
- Review mode allows users to replay games move by move.
- Modular code structure makes future improvements easier.
- Local execution means fast performance without relying on an internet connection.

Project Overview

Project Goal

- Develop a fully functional chess game using Python and Pygame featuring:
- Complete chess rules
- AI opponent
- Graphical User Interface
- Move validation
- Checkmate Detection



Problem Statement

The Problem

Many beginner chess players struggle to:

- Learn legal moves
- Understand good positions
- Practice without another player

Solution

A Chess AI Trainer that:

- Enforces legal moves
- Provides an AI opponent
- Displays checkmate detection
- Allows difficulty selection

Objectives

Success Criteria

-
- ✓ Complete a playable chess game
-
- ✓ Include AI opponent
-
- ✓ Difficulty Setting
-
- ✓ Checkmate Detection
-
- ✓ Legal move validation
-
- ✓ Allow game reset
-

Software Development Methodology



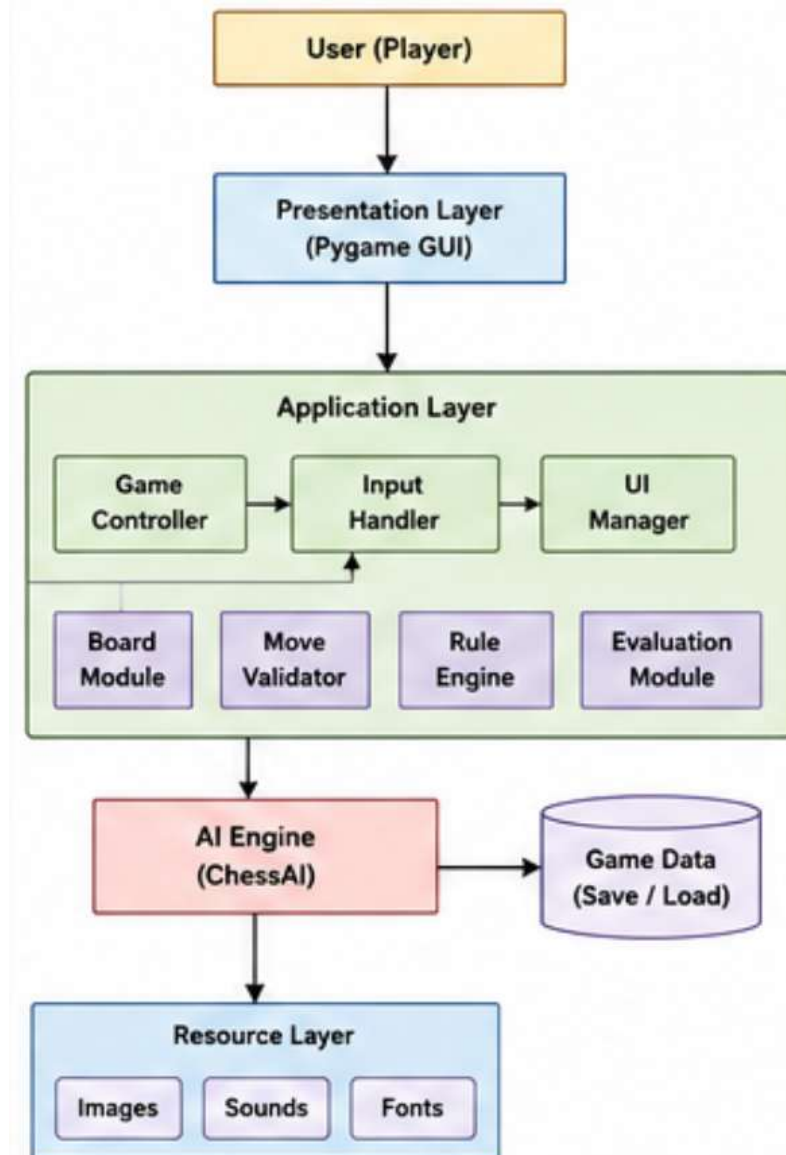
Waterfall Model

- Planning
- Analysis
- Design
- Development
- Testing
- Evaluation

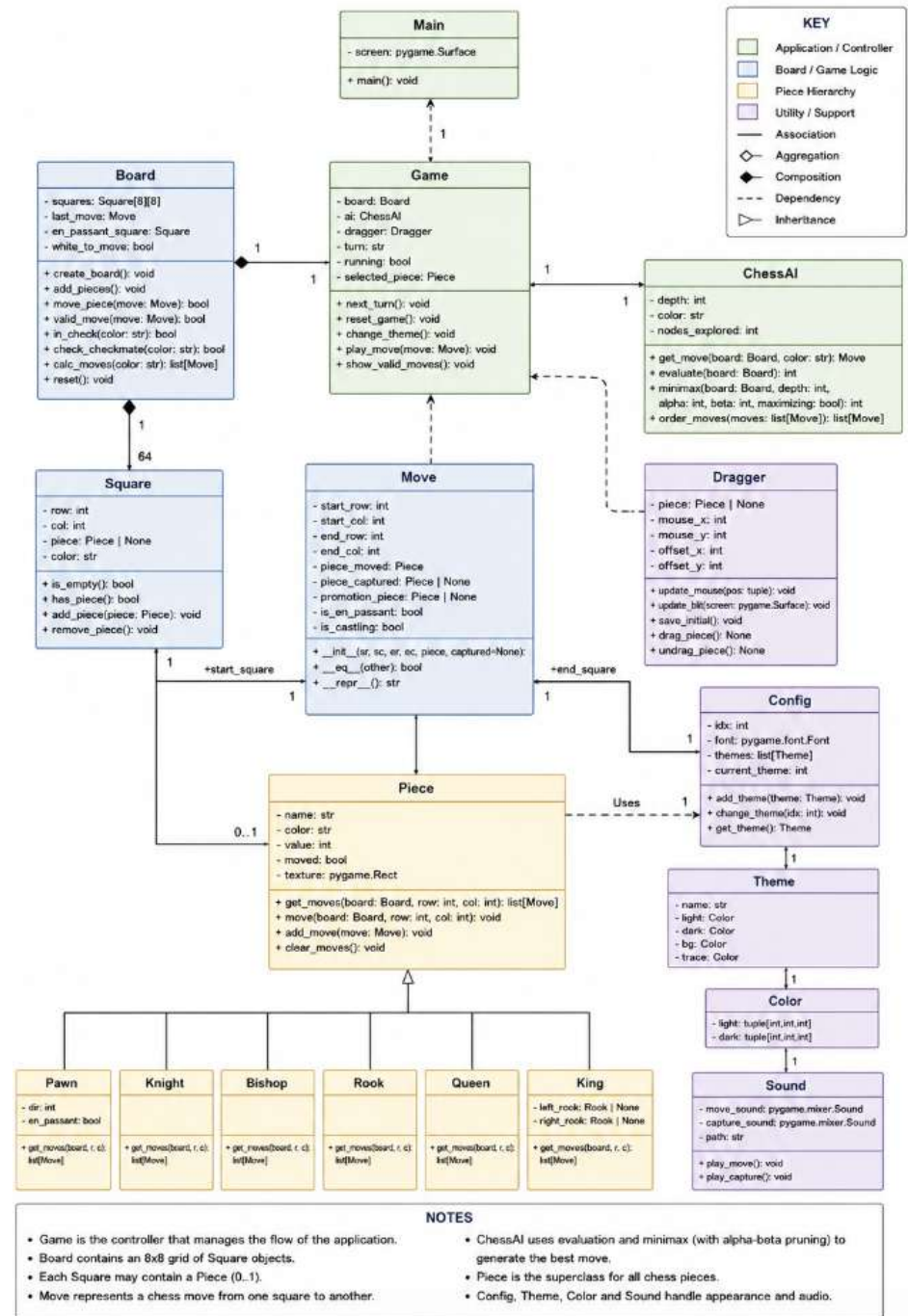
Technologies Used

Technology	Purpose
Python	Main language
Pygame	GUI
VS Code	Development
GitHub	Version control
Git	Tracking changes

System Architecture



UML Class Diagram



Core Algorithms

Move Validation

- Select piece
- Generate legal moves
- Validate move
- Execute move

AI Algorithm

- Generate moves
- Evaluate board
- Choose best move



Key Features

✓ Game
Reset

✓ Piece
movement

✓ Capturing

✓ Castling

✓ En Passant

✓ Promotion

✓ AI
Opponent

✓ Difficulty
Selection

✓ Checkmate
Detection

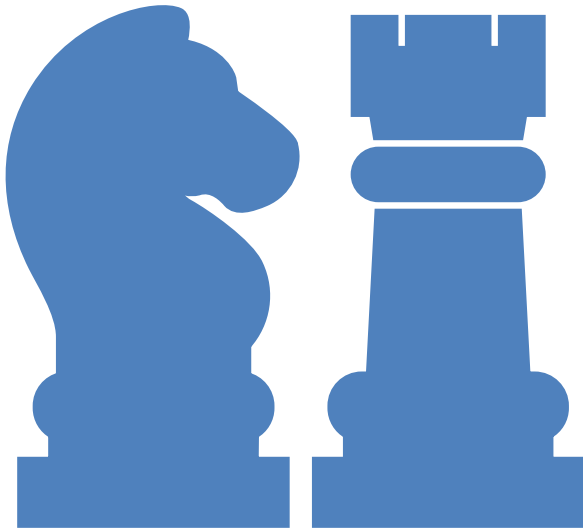
Testing

Test Case	Expected Result	Actual Result	Status
Pawn Movement	Pawn moves forward 1–2 squares from starting position	Worked correctly	✓ Pass
Knight Movement	Knight moves in L-shape pattern	Worked correctly	✓ Pass
Piece Capture	Opponent piece removed from board	Worked correctly	✓ Pass
AI Move Generation	AI responds after player move	Worked correctly	✓ Pass
Board Reset	Game returns to starting position	Worked correctly	✓ Pass
Pawn Promotion	Pawn becomes queen on final rank	Worked correctly	✓ Pass
Castling	King and rook move simultaneously	Worked correctly	✓ Pass
Evaluation Display	Board evaluation updates after moves	Worked correctly	✓ Pass

Purpose of Testing

Testing was conducted throughout development to ensure the chess game functioned correctly, responded to user actions, and produced valid AI moves without errors.

Testing Methods Used



Black Box Testing

The game was tested from the user's perspective without examining internal code. Inputs such as piece movements and menu selections were tested to verify correct outputs.

White Box Testing

Source code was inspected and debugged to verify move validation, AI decision-making, board updates, and game state management.

Boundary Testing

- Edge cases were tested including:
- Moving pieces at board edges
- Castling near board boundaries
- Pawn promotion on final rank
- Checkmate and stalemate situations

User Testing

- Friends and classmates tested the game and provided feedback regarding:
- Ease of use
- AI difficulty
- Interface design
- Gameplay functionality

Key Findings

✓ Piece movement was accurate

✓ AI responded automatically after player moves

✓ Game reset functionality worked correctly

✓ Evaluation system updated in real time

⚠ Checkmate and king movement bugs were discovered during testing and subsequently fixed

Code Documentation



The source code follows a 3:1 comment-to-code ratio where practical, ensuring each major block of code is explained.



Comments describe the purpose of classes, functions and complex algorithms rather than repeating obvious code.



Inline comments are used to explain AI decision-making, event handling and game logic.



Meaningful variable and function names reduce the need for excessive comments.



Consistent documentation makes the project easier to maintain, debug and extend by other developers.

AI Bugs

- Repeated moves

Checkmate Bugs

- False game endings

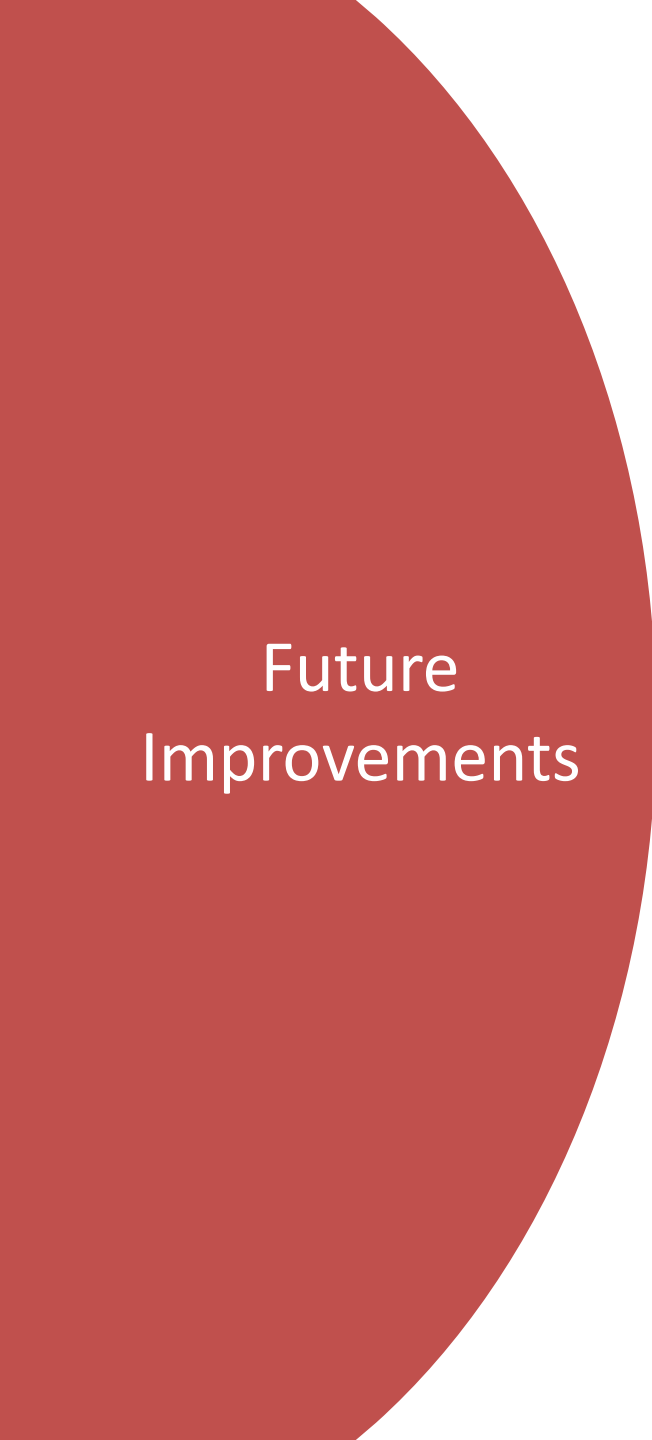
Challenges

King Movement Bugs

- Legal move issues

Tap-To-Move Bugs

- Input handling conflicts



Future Improvements

Minimax

- Smarter AI

Alpha-Beta Pruning

- Faster search

Undo Button

- Better usability

Move History

- Game analysis

Online Multiplayer

- Future expansion



Reflection

What I Learned

- Object-oriented programming
- AI development
- Debugging
- GitHub workflow
- Testing methodologies

Outcome

Project successfully met its objectives and produced a functional Chess AI Trainer.





Demonstration



Live Demo



Start game



Move piece



AI responds



Checkmate Detection